

Teaching a Small Model to Manage an AI Agent's Memory

Thesis proposal (updated October 2026)

The problem

AI agents built on large language models can only read a limited amount of text at once. In a long task, like a support chat that runs for days, the history soon grows past that limit, so something has to decide what the model keeps, what gets filed away, and what gets thrown out. Most systems use fixed rules like "keep the most recent messages," which are cheap but often throw away the one fact that ends up mattering.

Research questions

Overall: can a very small, separate model learn these memory decisions better than fixed rules, without retraining the language model itself?

RQ1. Under a tight memory budget, can a small learned controller beat fixed memory rules?

RQ2. What is the right way to learn from hindsight, when the controller can't see the future that its teacher sees?

RQ3. Does reinforcement learning (GRPO) add anything beyond imitation, and does it matter how it assigns credit to each decision?

RQ4. Does it hold up with real language models and real long conversations, and when is it worth it over a simple rule?

My approach

I built a test framework where a frozen language model answers questions while a separate "memory controller" decides what to keep, archive, delete or retrieve. The controller is tiny (about 27,000 parameters) and trains on a laptop. Because my tasks are simulated, I know afterwards which facts were needed, and I use that hindsight to build a teacher for the controller to copy, then try reinforcement learning (GRPO) on top.

What is new in this work

1. **A tiny controller kept separate from a frozen language model (RQ1, RQ4).** Most recent work retrains the language model itself; here only the small controller learns, so it can sit in front of any model. With Qwen2.5-7B reading, it matches showing the model everything while using 40% of the text.
2. **A "regret" teacher for copying hindsight (RQ2).** Copying an all-knowing teacher works badly (34% right at a tight budget), because the student can't see what it sees. Copying a teacher that accepts every choice that costs nothing gets 85%, beating every fixed rule I tested.
3. **Finding that search, not learning, was the bottleneck, and fixing it (RQ1, RQ3).** Most remaining mistakes were two-step questions whose second fact never showed up as an option. A simple "follow the clue" search raised accuracy from 85% to 92% (83% to 89% with Qwen2.5-7B), about half the gap to a perfect controller.
4. **Free per-question credit for GRPO (RQ3).** The simulator replays exactly, so each decision can be credited for its own question at no extra cost. GRPO adds about one point; the credit method made no difference.

What I will do next

1. (RQ4) Done: on real conversations the 7B model suffers "context rot" past about 3,000 prompt tokens, so keeping more memory does not help. A small learned picker that chooses which 5 search results to show beats the best simple rule (51% vs 47%) with half the prompt, across three training runs.
2. (RQ2, RQ3) Done: charging for kept tokens and reinforcement learning on the model's answers both add little, because what will be needed is unpredictable before the question. Next: let the picker choose how many results to show, then write up.

Why it matters

If a small, cheap model can manage memory well, any agent could use it without retraining a large model. If it can't, knowing why still helps anyone building agents that need to remember.

Addendum: formulations and diagrams

Notation follows the code in `memctl/`. Numbers are from EXPERIMENTS.md (Experiments D1, 9 and 10), scripted reader unless marked 7B; learned results are means of three training seeds.

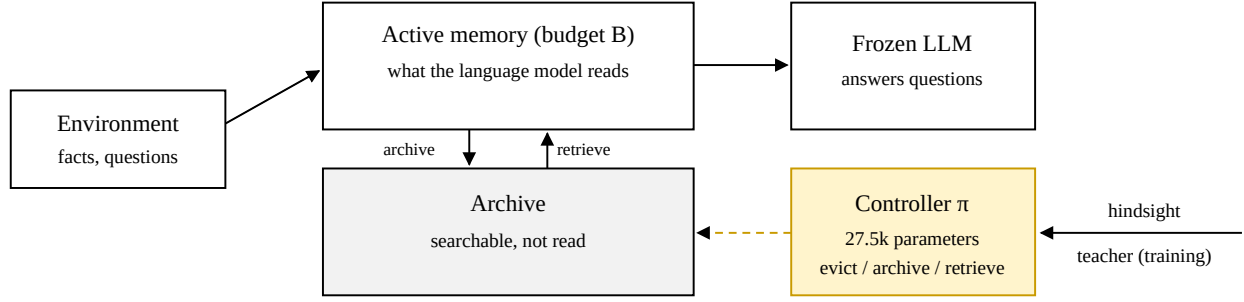


Figure 1. The controller decides what the frozen language model sees. During training only, a hindsight teacher labels its decisions.

A. Setup

At step t the memory state is $\mathbf{s}_t = (\mathcal{A}_t, \mathcal{R}_t)$: the active items, whose tokens must fit a budget $\sum_{i \in \mathcal{A}_t} \ell_i \leq B$, and the archive. The budget is a fraction of the whole history, $B = f \cdot L$ with $f \in \{2, 5, 10, 25\}\%$. When the active memory is over budget, the controller removes items one at a time, each by an operation $o \in \{\text{evict}, \text{archive}\}$, until it fits. When a question arrives, it may retrieve items from a shortlist $\mathcal{C}_t$ of the archive. The reward is the number of questions answered correctly, minus 0.1 for every forced fallback eviction:

$$R = \sum_t r_t, \quad r_t = \mathbb{1}[\text{question at } t \text{ answered correctly}] - 0.1 F_t.$$

B. Policy

A DeepSets scorer gives each candidate item i (features $\mathbf{x}_i$, plus mean and max pooled summaries of all candidates and global features $\mathbf{g}_t$) one logit per operation, $z_{i,o} = f_\theta(\mathbf{x}_i, \bar{\mathbf{x}}, \mathbf{g}_t)$, and a retrieval logit $z_{i,\text{ret}}$. Removals are a sequence of softmax draws without replacement over the remaining (item, operation) pairs, and retrievals are independent Bernoulli draws:

$$\pi_\theta(a_t | \mathbf{s}_t) = \prod_{j \in \mathcal{C}_t} \sigma(z_{j,\text{ret}})^{y_j} (1 - \sigma(z_{j,\text{ret}}))^{1-y_j} \prod_{k=1}^K \frac{\exp z_{(i_k, o_k)}}{\sum_{(i,o) \in M_k} \exp z_{(i,o)}},$$

where M_k is the set of pairs still available at the k -th removal.

C. Hindsight and the two teachers

A reference pass with unlimited memory gives every item's next need $\mathbf{n}_i(t)$: the first later step whose question uses it (∞ if never). The *oracle* teacher deletes never-needed items, then archives in order of furthest next need: one action per state. The *regret teacher* charges each (item, operation) its regret with the oracle playing on, and accepts every action of least regret:

$$\text{reg}(i, \text{evict}) = \mathbb{1}[\mathbf{n}_i(t) < \infty], \quad \text{reg}(i, \text{archive}) = c + \rho \mathbb{1}[\mathbf{n}_i(t) < \infty], \quad A^*(\mathbf{s}_t) = \arg \min_{(i,o)} \text{reg}(i, o),$$

with archive price $c = 0$ and retrieval risk $\rho = 0$ here. For a never-needed item both operations then cost 0. The oracle breaks that tie towards deletion, which a learner that cannot see the future cannot imitate, so it learns to delete needed items too. The student is trained by DAgger with a set-valued loss: at each removal along the teacher's sequence, the probability mass the policy puts on the accepted set; for retrieval, a cross-entropy against "needed right now":

$$\mathcal{L}_{\text{BC}}(\theta) = - \sum_k \log \sum_{a \in A_k^*} \pi_\theta(a | \mathbf{s}_k, M_k) - \sum_{j \in \mathcal{C}_t} \left[y_j^* \log \sigma(z_{j,\text{ret}}) + (1 - y_j^*) \log (1 - \sigma(z_{j,\text{ret}})) \right], \quad y_j^* = \mathbb{1}[\mathbf{n}_j(t) = t].$$

Teacher copied (2% budget)	Scripted reader	Qwen2.5-7B
Oracle teacher	0.335	0.320
Regret teacher	0.851	0.832
Best fixed rule (FIFO with archive)	0.759	0.741

D. GRPO

For each training episode (one seed and one budget) the policy samples a group of $G = 8$ rollouts with returns $R_1, \dots, R_G$. Every decision in rollout g gets the same group-standardised advantage, and the policy is updated with PPO's clipped objective and an entropy bonus, with no value network:

$$\hat{A}_g = \frac{R_g - \mu_G}{\sigma_G + \epsilon}, \quad \mathcal{J}(\theta) = \mathbb{E} \left[\min(\rho_t \hat{A}_g, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_g) \right] + \beta \mathcal{H}(\pi_\theta), \quad \rho_t = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)},$$

with $\epsilon = 0.2$, $\beta = 0.01$, 4 epochs per batch, and $\hat{A}_g = 0$ when the group is tied.

E. Modifications of GRPO tested

1. **Batch scale, per budget** (after Dr. GRPO and Lite PPO). Centre within the group, but scale by the spread of the whole batch within the same budget stratum k , so nearly tied groups are not blown up to ± 1 :

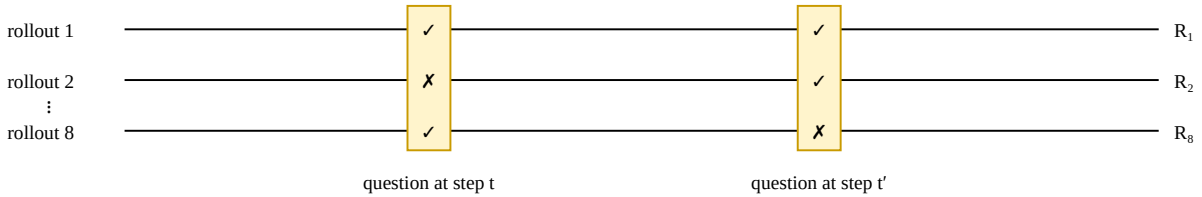
$$\hat{A}_g = \frac{R_g - \mu_G}{\sigma_{B,k} + \epsilon}, \quad \sigma_{B,k}^2 = \frac{1}{|B_k|} \sum_{g \in B_k} (R_g - \mu_{G(g)})^2.$$

2. **Best-of- G imitation** (RAFT), a filtered-imitation baseline: $\hat{A}_g = \mathbb{1}[R_g = \max_{g'} R_{g'} \text{ and unique}]$.
3. **Per-question credit**. All G rollouts of a group replay the same seed, so they meet the same question at the same step t . A decision at t also gets its rollout's reward at t relative to the group's mean reward at t , at no extra cost:

$$\hat{A}_{g,t} = \hat{A}_g + \omega(r_t^{(g)} - \bar{r}_t), \quad \bar{r}_t = \frac{1}{G} \sum_{g'} r_t^{(g')}, \quad \omega = 1.$$

4. **Expert kept in the loss** (after CHORD and SRFT): $\mathcal{L} = -\mathcal{J}(\theta) + \lambda \mathcal{L}_{\text{BC}}(\theta)$ with $\lambda = 0.5$, the regret teacher labelling the learner's own states.

One group: 8 replays of the same episode (seed, budget)



GRPO credits every decision in a rollout with one number, from $R_1 \dots R_8$. Per-question credit also compares the rollouts at each question (columns).

Figure 2. Episode-level against per-question credit in one GRPO group.

F. The follow-the-clue search

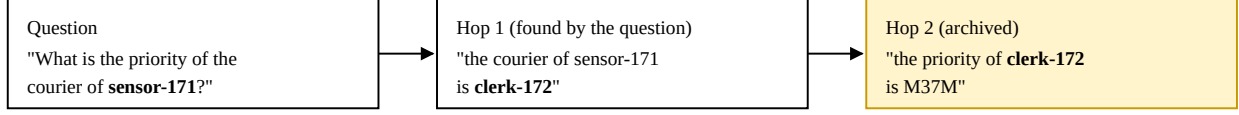
The controller can only retrieve from a shortlist: the top 8 archive items for the question q by BM25. In a two-hop question the second fact names an entity the question never mentions, so it is rarely on that list (Figure 3). The search adds a second pass. Let $W(x)$ be the content words of x and $\text{df}(w)$ the number of memory items containing w .

1. Anchor: the question's rarest word in memory, $w^* = \arg \min_{w \in W(q), \text{df}(w) > 0} \text{df}(w)$ (what the question names).
2. Seeds: the best BM25 matches to q in active memory or the archive that contain w^* and add new words; the first $m = 2$ are used.

3. For each seed s , a second query: the question's words the seed lacks, plus the seed's $c = 3$ rarest new words (the bridge entity):

$$q'(s) = (W(q) \setminus W(s)) \cup \text{rarest}_c(W(s) \setminus W(q)).$$

4. The top $k = 4$ archive items by $\max_s \text{BM25}(q'(s), \cdot)$ join the shortlist, with a feature marking them.



Question search: hop 2 shares only "priority" with the question, like hundreds of archived facts.

Second query from the seed: "priority ... clerk-172" puts it first.

Reach of the regret policy's 195 hop-2 misses (seed 0): 102 in the question top 8, 184 in the clue search's top 4.

Figure 3. Why two-hop questions failed, and how the second search reaches the missing fact.

G. Results of the changes (2% budget, scripted reader, 3 seeds each)

Controller	Task success	Paired difference (95% CI)
Regret imitation	0.851	
+ follow-the-clue search	0.917	+0.067 [+0.054, +0.079] vs regret imitation
+ plain GRPO	0.928	+0.011 [+0.004, +0.018] vs follow-the-clue
+ GRPO, batch scale per budget	0.930	+0.013 [+0.008, +0.018]
+ per-question credit	0.932	+0.014 [+0.009, +0.020]
+ expert kept in the loss	0.929	+0.012 [+0.007, +0.016]
Hindsight oracle (upper reference)	0.966	

With Qwen2.5-7B as the reader (Experiment 11a, 100 episodes, same seeds):

Controller (Qwen2.5-7B)	2%	10%	Paired difference at 2% (95% CI)
FIFO with archive	0.741	0.787	
Regret imitation	0.832	0.884	
+ follow-the-clue search	0.889	0.917	+0.057 [+0.045, +0.069]
Hindsight oracle	0.938	0.949	
Full context (no budget)	0.898		

All GRPO variants were run at matched experience (1,440 episodes). None differs from plain GRPO at 2% (largest +0.003 [−0.001, +0.008]). Before the search fix, GRPO and best-of-8 imitation each added under one point (0.859 and 0.861 against 0.851).